

Card 1: Shard-per-Core

One OS thread pinned per CPU core. Shares nothing between cores, eliminating locks and cache line bounces.

Card 5: SPSC Cross-Core Ring

Coordinates cross-core message exchanges using lock-free single-producer single-consumer ring buffers.

Card 9: Micro-Scheduler

Maintains FIFO task queues inside Reactors, scheduling fiber executions using CPU priority weights.

Card 13: Direct I/O DMA

Bypasses the OS page cache using O_DIRECT, reading and writing directly to storage controllers via DMA.

Card 2: CPU Core Pinning

Pin threads using sched_setaffinity. Prevents context switches and keeps cache lines clean of task migrations.

Card 6: Future Promise

Single-threaded futures contain zero locks. Cross-shard modifications are forbidden to prevent data races.

Card 10: Exception Propagation

Captures errors inside futures using handle_exception, and manages resource releases via finally.

Card 14: Disk I/O Fair Queue

Balances IOPS and storage bandwidth across shards to prevent individual cores from starving disk access.

Card 3: Lock-Free Allocator

Preallocates memory per core. Benchmarks request memory from thread-local regions, avoiding global heap locks.

Card 7: Chained Continuation

Chains execution paths using .then() operators, utilizing move semantics to forward results without copies.

Card 11: io_uring Epoll

Uses io_uring queues to bypass system call context switches. Falls back to Epoll loops on older kernels.

Card 15: DPDK User-space Stack

Integrates DPDK to intercept network interface packets directly in user space, bypassing Linux protocols.

Card 4: Cooperative Fibers

Fibers yield control to the Reactor upon encountering I/O waits. Synchronous blocks will stall execution.

Card 8: C++20 Coroutines

Translates co_await operations to continuation closures in compilation, boosting async readability.

Card 12: Zero-Copy Packets

Uses fragmented packet memory. Shares references instead of copying data payloads across targets.

Card 16: Lock-Free Shared Atom

Restricts shared variables to std::atomic and memory barriers to limit cache coherency bus storms.

Card 17: Read-Copy-Update RCU

Provides lock-free read execution by copying data updates and swapping pointers atomically via CAS.

Card 21: Sharded Services

Instantiates sharded service objects across cores, coordinating data merges via `sharded::map_reduce`.

Card 25: Stalled Watchdog

Triggers watchdog warnings with stack backtraces when tasks monopolize Reactor cores for more than 500us.

Card 18: Coordinate Barriers

Synchronizes startup and shutdown steps across shards asynchronously using `smp::invoke_on_all`.

Card 22: Connection Rate Limit

Monitors active connection pools, dropping or redirecting queries during core load spikes.

Card 26: Stack Reconstruction

Links active async context spans in debug modes to logically reconstruct calling paths across future chains.

Card 19: Cache Line Alignment

Forces 64-byte alignment on shared structures using `alignas(64)` to prevent core false sharing.

Card 23: Backpressure Flow

Throttles upstream sockets when internal buffers reach capacity limits, preventing memory exhaustion.

Card 27: Telemetry Counters

Collects Reactor counters on processed tasks, pending futures, and memory allocation events for analysis.

Card 20: NUMA-Aware Memory

Allocates memory local to the thread's CPU socket, preventing latency penalties from cross-socket accesses.

Card 24: Zero-Copy Parsers

HTTP parser and JSON compiler read data directly from packet buffers, skipping allocation copies.

Card 28: Compiler Optimization

Uses LTO and native target compiler flags to compile inlined futures and optimize execution latency.